

Acer Wave 7 MESH Router Remote Code Execution

30.04.2026

Author: Pap Gergo

Description

The vulnerabilities described here and the methods used to exploit them are the result of research conducted in 2025. By the way, this is what led to the permanent deletion of my LinkedIn account... because I published a video demonstrating the exploitation of the vulnerability. Of course, there was no technical write-up included, but apparently it still violated the platform's policies — or maybe I simply shouldn't have tagged Acer in the post (who knows!).

At that time, I had not yet reported the vulnerability to Acer, and to be honest, I wasn't planning to report it either. Why? Very simple... No bounty, no submit, no responsible disclosure. Simple as that.

To elaborate a bit further, lately I've noticed that some vendors have become far too comfortable with receiving vulnerability reports from enthusiastic researchers for free. Maybe one day I'll go deeper into how I think this mindset is pushing the entire security industry in the wrong direction... but let's leave that aside for now.

So, Acer Wave 7 router RCE...

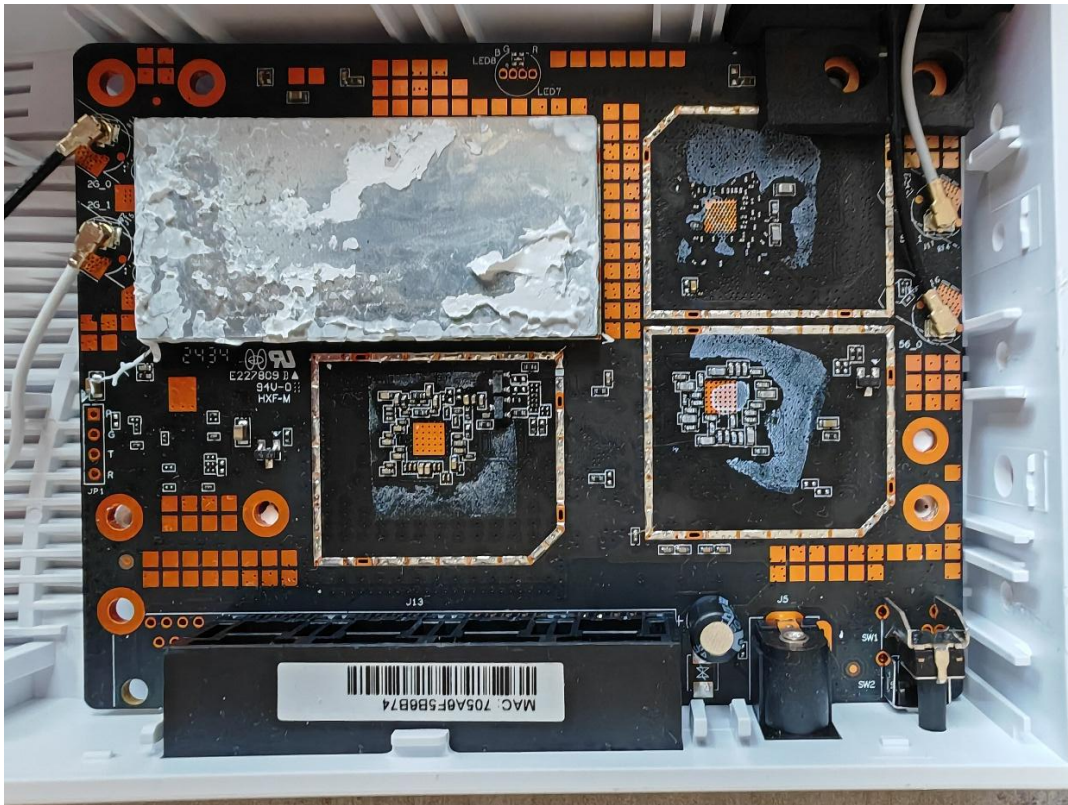
To achieve remote code execution and gain full control over the router, I used two vulnerabilities.

During the initial phase of the research...

As with pretty much every IoT vulnerability research project, I started by trying to obtain the firmware... which, of course, was not available for direct download.

At this point, one option would have been to start monitoring the network traffic and observe which server the device contacts or downloads the firmware from during the update process. But this time I skipped that part. Instead, I figured it would be more interesting to tear the device apart and see what kind of hardware was hiding beneath the plastic casing — maybe the manufacturer had kindly left behind a fully functional UART port with full console access...

SPOILER: they did...



The UART console could be accessed using the same password as the one used for the web interface, together with the root user account. This provided full access to the entire filesystem, configuration files, binaries — essentially everything...

```
T7c login: root
Password:

BusyBox v1.35.0 (2025-03-15 03:03:49 UTC) built-in shell (ash)
"the quieter you become, the more you are able to hear"

acer T7c 1.01.000055

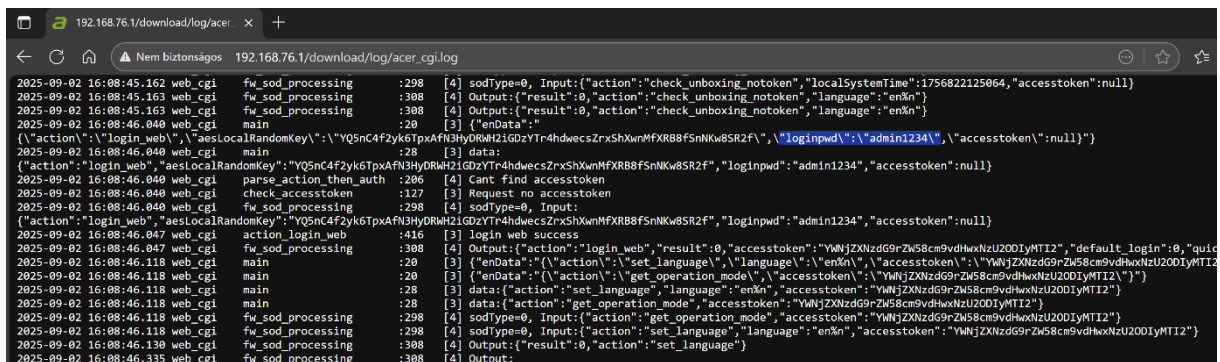
root@T7c:~#
```

```
root@T7c:~# cd www/
root@T7c:/www# ls
cgi-bin      download    fonts       index.html  version.json
css          favicon.ico img          js
root@T7c:/www# cd cgi-bin/
root@T7c:/www/cgi-bin# ls
common_cgi  ipv6_cgi   security_cgi upload       web_cgi
ddns_cgi    lan_cgi    sse_cgi     vpn_cgi     wifi_cgi
firewall_cgi qos_cgi     system_cgi  wan_cgi
root@T7c:/www/cgi-bin#
```

Vulnerability #1

Broken access control:

Due to improper permission settings, the acer_cgi.log file is accessible without authentication at "http://192.168.76.1/download/log/acer_cgi.log" This file contains the login credentials used for the web interface (and telnet), which can be used to log in to the device's web management interface.



```
2025-09-02 16:08:45.162 web_cgi fw_sod_processing :298 [4] sodType=0, Input:{"action":"check_unboxing_notoken","localSystemTime":1756822125064,"accessToken":null}
2025-09-02 16:08:45.163 web_cgi fw_sod_processing :308 [4] Output:{"result":0,"action":"check_unboxing_notoken","language":"en\u00c3\u00a9n"}
2025-09-02 16:08:45.163 web_cgi fw_sod_processing :308 [4] Output:{"result":0,"action":"check_unboxing_notoken","language":"en\u00c3\u00a9n"}
2025-09-02 16:08:46.040 web_cgi main :20 [3] {"enData":...
{"action":"login_web","aesLocalRandomKey":"YQ5nC4f2yk6TpxAFN3HyDRWHz2iGdzYTr4hdwecsZrxShXwnMFXRB8fSnNKw8SR2f","loginpwd":"admin1234","accessToken":null}
2025-09-02 16:08:46.040 web_cgi main :28 [3] data:
{"action":"login_web","aesLocalRandomKey":"YQ5nC4f2yk6TpxAFN3HyDRWHz2iGdzYTr4hdwecsZrxShXwnMFXRB8fSnNKw8SR2f","loginpwd":"admin1234","accessToken":null}
2025-09-02 16:08:46.040 web_cgi parse_action_then_auth :206 [4] Cant find accessToken
2025-09-02 16:08:46.040 web_cgi check_accessToken :127 [3] Request no accessToken
2025-09-02 16:08:46.040 web_cgi fw_sod_processing :298 [4] sodType=0, Input:
{"action":"login_web","aesLocalRandomKey":"YQ5nC4f2yk6TpxAFN3HyDRWHz2iGdzYTr4hdwecsZrxShXwnMFXRB8fSnNKw8SR2f","loginpwd":"admin1234","accessToken":null}
2025-09-02 16:08:46.047 web_cgi action_login_web :416 [3] login web success
2025-09-02 16:08:46.047 web_cgi fw_sod_processing :308 [4] Output:{"action":"login_web","result":0,"accessToken":"YwNjZXNzdG9rZW58cm9vdHwNzU200IyMTI2","default_login":0,"quick_login":0}
2025-09-02 16:08:46.118 web_cgi main :20 [3] {"enData":...
{"action":"set_language","language":"en\u00c3\u00a9n","accessToken":"YwNjZXNzdG9rZW58cm9vdHwNzU200IyMTI2"}
2025-09-02 16:08:46.118 web_cgi main :28 [3] data:{"action":"set_language","language":"en\u00c3\u00a9n","accessToken":"YwNjZXNzdG9rZW58cm9vdHwNzU200IyMTI2"}
2025-09-02 16:08:46.118 web_cgi fw_sod_processing :298 [4] sodType=0, Input:{"action":"set_language","language":"en\u00c3\u00a9n","accessToken":"YwNjZXNzdG9rZW58cm9vdHwNzU200IyMTI2"}
2025-09-02 16:08:46.130 web_cgi fw_sod_processing :308 [4] Output:{"result":0,"action":"set_language"}
2025-09-02 16:08:46.335 web_cgi fw_sod_processing :308 [4] Output:
```

Vulnerability #2

Hardcoded cryptography key:

The Acer Wave Connect 7 router's web interface allows the creation of backup files. These backups are AES-encrypted binaries. However, the upload.cgi binary, which is responsible for processing backup files, contains the encryption key hardcoded. Possession of this key enables decryption of the backup file.

```
24 | sprintf(acStack_824,"%s%s","/tmp/", "backup_settings.tar.gz.aes");
25 | rename(param_1,acStack_824);
26 | sprintf(acStack_424,"%s%s","/tmp/", "backup_settings.tar.gz");
27 | sprintf(acStack_c24,"openssl enc -d -aes128 -salt -pbkdf2 -in %s -out %s -pass pass:*DtAknrG3d",
```

Exploitation

The exploitation naturally relied on chaining the two vulnerabilities together.

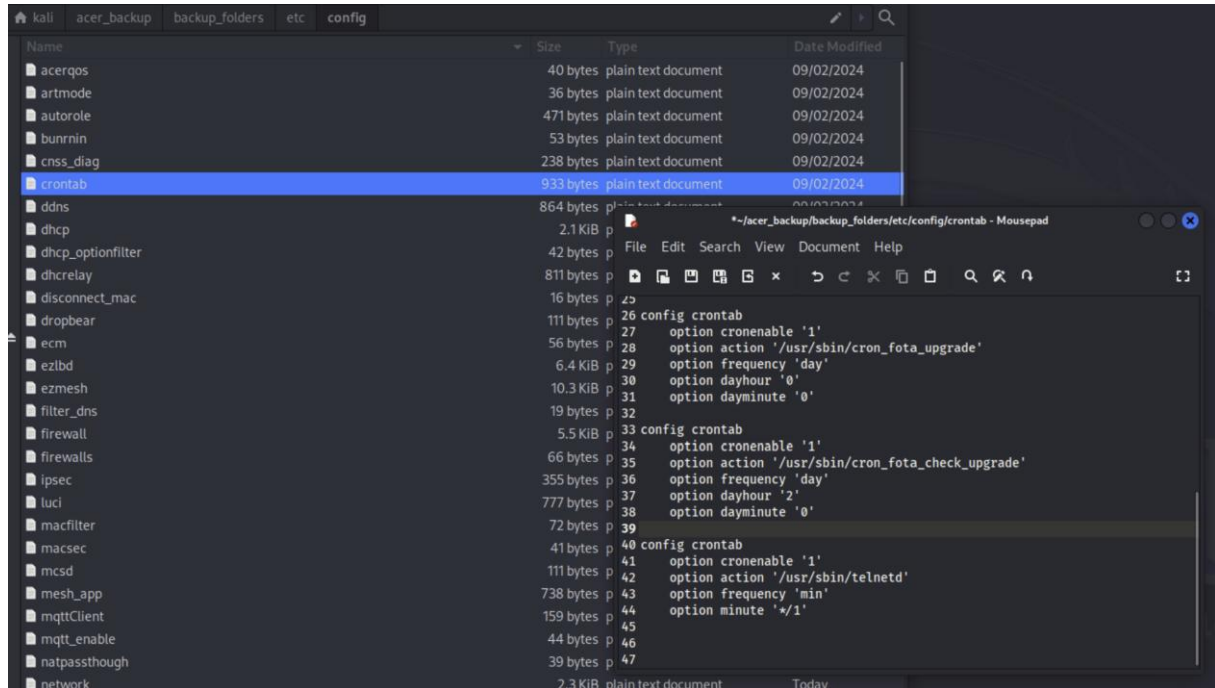
The first vulnerability was fairly straightforward: the log file was accessible without authentication and contained login credentials that could be used to access the web management interface.

The web interface also provided the ability to download and upload backup files. And this is where things became interesting: the backup file itself was encrypted, however, the upload.cgi binary — responsible for handling the backup upload process on the router — contained the encryption key used for protecting those backups.

With possession of the key, it became possible to decrypt the backup file, freely modify its contents, and then re-encrypt it using the known key. After that, the modified backup could simply be uploaded back to the router.

Modify the backup file and get telnet access

The decrypted backup file actually contained numerous directories and configuration files, but the most interesting part for us was the crontab file located in the /etc/config directory.



This file contained scheduled tasks, such as checking for firmware updates.

The file could simply be modified so that it periodically launched the Telnet daemon...

So, in practice, the exploitation process looked like this:

1. Download the log file containing the credentials for the web management interface.
2. Use those credentials to login to the web interface and download the backup file.
3. Modify the crontab file so it starts the Telnet daemon.
4. Upload the modified backup file back to the router.
5. Enjoy root-level access.



+1) In the case of WAN-side exploitation, modify the firewall rules as well.